

WOLFENSTEIN 3D → STM32F411CEU6

Port Analysis, Strip List & Optimisation Guide

2.4" SPI TFT Display (ILI9341 / ST7789) · Bare-Metal C · ARM Cortex-M4

1. Hardware Constraints at a Glance

Before any line of code is written, every porting decision must be measured against these hard limits:

Parameter	Value / Implication
CPU	ARM Cortex-M4 @ 100 MHz (STM32F411CEU6)
FPU	Single-precision FPU — use float, avoid double
Flash	512 KB — entire game code + stripped assets must fit
SRAM	128 KB — framebuffer + stack + game state must share this
Display	240×320 px, 16-bit RGB565, SPI bus
Render target	240×320 = 76,800 px × 2 bytes = 150 KB (FULL frame impossible)
Practical framebuffer	Half-frame strip: 240×160 × 2 = 75 KB (fits with care)
SPI speed (max)	~42 MHz — full frame DMA push ≈ 14 ms (~71 fps ceiling)
Wolf3D original res	320×200 px — must rescale to 240×160 (game area)
Wolf3D original RAM	~640 KB conventional + EMS — 5× what we have
Wolf3D original CPU	286/386 @ 8–33 MHz — similar speed, totally different arch
No OS	Bare-metal / FreeRTOS only — no malloc after init, no file system

2. Original Wolf3D Source Architecture

The WOLFSRC directory contains ~30 C and assembly files, organized into these subsystems. Understanding each is required before deciding what to strip.

2.1 Core Raycaster (WL_DRAW.C, WL_SCALE.C, WL_AGENT.C)

- Column-by-column ray casting loop — one ray per screen column
- Integer fixed-point math using 16.16 format with lookup tables (FIXTABLES)
- Wall scaling via dynamically compiled x86 machine code routines (ScaleShape / HitHorizBorder) — these are purely PC/DOS-specific and must be completely rewritten
- Sprite clipping and depth-sorted sprite drawing (WL_SCALE.C)
- Floor and ceiling drawn as solid colours only — no texture mapping

2.2 Map / Level System (WL_MAP.C, MAPHEAD.H, GAMEMAPS.*)

- 64×64 tile maps, 2-byte tiles, stored in id's proprietary RLEW + Carmack-compressed format
- Three map planes: walls, objects, floor/info — only the wall plane is critical for rendering
- Map data is loaded from disk via the ID_CA cache manager — not applicable on bare-metal
- Total map data for all 60 levels: ~400 KB compressed (too large for Flash solo)

2.3 Asset Pipeline (ID_CA.C, ID_VH.C, WL_GAME.C)

- ID Cache Manager (ID_CA) handles all asset I/O: textures, sprites, sounds, maps
- Assets are packed in VGAGRAPH, VSWAP, AUDIOT, GAMEMAPS files — all DOS-era formats
- VGA palette (256 colours) is converted to RGB at runtime
- All file I/O goes through DOS interrupt 21h — must be fully replaced

2.4 Video Output (ID_VH.C, ID_VL.C, WL_DRAW.C)

- Targets Mode 13h VGA: 320×200, 256-colour, byte-mapped linear framebuffer at 0xA0000
- Palette remapping, color cycling, screen wipes (fades) all done in software
- No double-buffering in the original — screen is written directly

2.5 Sound System (ID_SD.C, AUDIOT.*)

- AdLib OPL2 FM synthesis (port I/O 0x388) for music
- PC speaker driver for sound effects
- IMF music format for background music
- All of this is 100% PC-specific — strip entirely for a first port

2.6 Input System (ID_IN.C)

- Keyboard via BIOS int 9h interrupt handler
- Joystick via ISA game port (0x201)
- Mouse via int 33h DOS mouse driver

- Replace entirely with: analogue joystick (ADC1 dual-channel DMA scan) + joystick click switch (GPIO) + BlackPill KEY button on PA0 (GPIO)

2.7 Game Logic (WL_GAME.C, WL_PLAY.C, WL_ACT1.C, WL_ACT2.C, WL_BOSS.C)

- Player movement, collision, door/elevator state machines
- Enemy AI: 8-state FSM per actor, path-finding via simple line-of-sight
- Item pickup, score, lives, end-game triggers
- This is the most portable layer — keep with minimal changes

2.8 Memory Manager (ID_MM.C, ID_PM.C)

- Segment-based 16-bit DOS memory manager with EMS/XMS expansion
- Page manager for asset streaming from disk
- Completely incompatible with ARM flat-address model — replace with static pools

3. What to Strip — and Why

3.1 Strip Entirely — No Salvageable Code

Source Files	Subsystem	Reason
ID_SD.C, AUDIOT.*	Sound system	OPL2, PC speaker, IMF — all ISA bus hardware. No analogue equivalent targeted.
ID_IN.C (most)	Input (DOS)	int 9h, int 33h, game port — all x86 BIOS/DOS. Replace with STM32 GPIO.
ID_MM.C, ID_PM.C	DOS memory mgr	16-bit segment model, EMS/XMS. Incompatible with ARM flat memory map.
ID_CA.C (file I/O)	Cache / file I/O	DOS int 21h file access. Replace with Flash read or SPI-SD if used.
WL_SCALE.C (x86 asm)	Dynamic code gen	Self-modifying x86 asm scaling routines. Replace with integer C loops.
ID_VL.C (VGA out)	VGA driver	Mode 13h palette VGA. Replace with SPI TFT driver (ILI9341/ST7789).
SIGNON.C, WL_INTER.C	Intro/intermission	Full-screen splash & story screens. Not renderable — strip for MVP.
WL_MENU.C (most)	DOS UI menus	VGA text menus with mouse. Strip to minimal start/pause if needed.
SCRNSAVE.C	Screen saver	Irrelevant on embedded — remove.
WL_DEBUG.C	Debug overlay	DOS-specific debug display. Remove or stub.

3.2 Strip Partially — Keep Logic, Replace Platform Layer

- WL_DRAW.C — Keep raycasting math; rewrite the pixel-write path to target RGB565 line buffer
- WL_GAME.C — Keep game FSM; strip save/load, cheat menus, episode selection
- WL_ACT1/2.C — Keep enemy AI state machines; strip episode-specific boss behaviours initially
- ID_CA.C — Strip file I/O; keep decompression routines (RLEW, Carmack) as asset unpacker
- WL_MAP.C — Keep tile lookup; strip map loading — bake one map into Flash

3.3 Asset Reduction — The Biggest Win

- **Original VSWAP.WL6 contains 1,544 textures/sprites at 64×64, 8bpp — ~6 MB total**
 - Target budget: 200 KB in Flash for textures → keep ~50 walls + ~20 sprites, re-encoded as RGB565
 - Map budget: one or two levels only — ~8 KB each uncompressed
 - Convert all 256-colour palettised textures offline to 4-bit or RGB565 before flashing
 - Tools to use: Python + Pillow to batch-extract and convert VSWAP assets offline
-

4. What to Optimise — Detailed Strategies

4.1 Raycaster Rendering Pipeline

4.1.1 Resolution & Framebuffer Strategy

A full 240×320 RGB565 framebuffer requires 150 KB — more than the total SRAM. Use horizontal strip rendering:

- Render in strips: 240 px wide × N rows tall (e.g. N=40 rows = 18.75 KB per strip)
- Render one strip, DMA-push to display via SPI, then render next strip — pipeline CPU and SPI
- Effective game viewport: 240×160 (matching Wolf3D's 320×168 aspect after 3:2 crop)
- Status bar (remaining 240×160 rows): drawn from a small static bitmap, pushed once per frame

4.1.2 Fixed-Point Math

- Wolf3D already uses 16.16 fixed-point — this maps directly to ARM 32-bit integers
- Replace all float sin/cos lookups with PROGMEM (Flash) tables of fixed-point values
- Use `__builtin_clz()` for fast integer log2 in distance calculations
- Enable FPU in CPACR register — use for angle interpolation only, not inner loop

4.1.3 Wall Scaling — New Integer Loop

The original dynamically compiled scale routines must be rewritten as a simple C loop:

```

void DrawWallColumn(uint16_t *lineBuf, const uint8_t *tex, int yTop, int yBot, int
texCol) {
    int drawHeight = yBot - yTop;
    int texStep = (64 << 16) / drawHeight; // fixed-point step
    int texPos = 0;
    for (int y = yTop; y < yBot; y++) {
        lineBuf[y * 240 + x] = palette565[tex[texCol + ((texPos >> 16) & 63) * 64]];
        texPos += texStep;
    }
}

```

Compile with -O2 and -funroll-loops. The Cortex-M4 pipeline handles this in ~3 cycles/pixel.

4.2 Memory Layout — Static Allocation Only

Never call malloc() after system init. Lay out SRAM statically:

Region	Size	Contents
Framebuffer strip A	~18 KB	Active render strip (DMA source)
Framebuffer strip B	~18 KB	Next render strip (CPU writes while A is sent)
Texture cache	~40 KB	Hot wall textures decoded to RGB565
Sprite buffer	~8 KB	Per-frame sprite column mask + pixel buffer
Game state	~12 KB	Player, actors (max 32), doors, tiles, score
Map data	~8 KB	One level decompressed tile array
Stack + misc	~24 KB	IRQ stacks, FreeRTOS TCBs if used

Total: ~128 KB — exactly at limit. Profile and trim aggressively.

4.3 SPI / DMA Display Driver

- Use SPI1 at APB2 / 2 = 42 MHz max — connect TFT CS, DC, RST to dedicated GPIOs
- Configure DMA2 Stream 3 (SPI1_TX) for memory-to-peripheral, circular-off, 16-bit words
- In the DMA half-complete IRQ: begin rendering next strip into the idle buffer
- In the DMA complete IRQ: swap buffers, trigger next DMA transfer immediately
- Full 240×320 frame over SPI at 42 MHz: $(240 \times 320 \times 2 \times 8) / 42e6 \approx 29 \text{ ms} \approx 34 \text{ fps}$ maximum
- Viewport-only (240×160): $\approx 14 \text{ ms}$ — allows ~71 fps ceiling before logic overhead

4.4 Flash Layout — Asset Storage

- Place all texture data in a dedicated .rodata section marked `__attribute__((section(".textures")))`
- Link script: place .textures after .text, align to 4-byte boundary
- Access textures directly via pointer — no copy to RAM needed if bus is fast enough
- Use `__attribute__((optimize("O3")))` on the raycaster inner loop only
- Enable instruction cache (ICACHE) and data cache (DCACHE) via FLASH_ACR

4.5 Game Loop Timing

- Use TIM2 (32-bit) as game tick source — configure for 70 Hz (14.3 ms period) to match original
- Run game logic in the main loop gated by a volatile tick flag set in TIM2 IRQ
- Raycasting and DMA push run concurrently — target <14 ms total per frame
- Measure with TIM5 free-running counter and DWT cycle counter (CoreDebug->CYCCNT)

5. Prioritised To-Do List

All tasks below, ordered by dependency and priority. Complete Phase 1 before touching Phase 2.

ID	Task	Priority	Effort	Category
T01	Set up STM32CubeIDE project, enable FPU, configure clocks to 100 MHz	HIGH	0.5 d	Platform Setup
T02	Write ILI9341/ST7789 SPI TFT driver with DMA (init + draw_rect + push_strip)	HIGH	1 d	Display Driver
T03	Implement double-buffer strip DMA with half/full-complete IRQs	HIGH	1 d	Display Driver
T04	Build offline Python tool to extract & convert VSWAP textures → RGB565 C arrays	HIGH	1 d	Asset Pipeline
T05	Build offline Python tool to extract & decompress one GAMEMAPS level → C array	HIGH	0.5 d	Asset Pipeline
T06	Port WL_DRAW.C raycaster: remove x86 asm, write ARM fixed-point ray loop	HIGH	3 d	Core Renderer
T07	Write integer wall-column scaler (replace WL_SCALE.C dynamic code gen)	HIGH	1 d	Core Renderer
T08	Implement RGB565 palette conversion from Wolf3D 256-colour palette	HIGH	0.5 d	Core Renderer
T09	Port player movement, rotation, collision (WL_PLAY.C / WL_AGENT.C)	HIGH	2 d	Game Logic
T10	Configure ADC1 dual-channel DMA scan for joystick X/Y axes (PA1, PA2); configure PA0 (BlackPill KEY) and joystick click GPIO as digital inputs with debounce; write input abstraction layer	HIGH	1 d	Input
T11	Port door open/close state machine (WL_ACT1.C)	MED	1 d	Game Logic
T12	Port static sprite rendering (guards/items) with depth sorting	MED	2 d	Core Renderer
T13	Port basic enemy AI FSM (WL_ACT2.C) for one guard type	MED	2 d	Game Logic
T14	Implement minimal HUD: health + ammo drawn as static 4-bit tiles	MED	1 d	UI
T15	Profile raycaster with DWT cycle counter, apply -O2 / loop unroll	MED	1 d	Optimisation
T16	Implement texture cache — pre-decode 16 hot textures to RGB565 in SRAM	MED	1 d	Optimisation
T17	Add second map level + level-transition trigger	MED	1 d	Game Logic

ID	Task	Priority	Effort	Category
T18	Implement simple PWM buzzer for footstep / shoot sound cues	LOW	1 d	Audio
T19	Add FreeRTOS (optional) — separate render task from logic task	LOW	1 d	Platform
T20	Add SPI MicroSD for future map/asset streaming	LOW	2 d	Asset Pipeline

6. Elaborated Implementation Steps

Step 1 — Project Setup & Clock Configuration (T01)

This is your foundation. Every subsequent decision depends on knowing exact clock speeds and memory regions.

1. Create a new STM32CubeIDE project targeting STM32F411CEU6 (UFQFPN48 package).
2. In CubeMX clock tree: set HSE = 25 MHz (or use HSI 16 MHz), PLL M=25 N=200 P=2 → SYSCLK = 100 MHz.
3. Enable FPU: in startup code or SystemInit(), set CPACR bits CP10/CP11 to 0b11.
`SCB->CPACR |= (0xF << 20); // Enable CP10 + CP11 (FPU full access)`
4. Enable Flash prefetch, ART accelerator, and wait states (FLASH_ACR = PRFTEN | ICEN | DCEN | LATENCY_3WS).
5. Configure linker script (STM32F411CEUX_FLASH.ld): define .textures region after .text, .framebuf region in SRAM.
6. Verify with a blink test at 100 MHz using TIM2 and DWT counter.

Step 2 — SPI TFT Driver with DMA (T02, T03)

This is the most hardware-specific work. Get pixels on screen before touching any Wolf3D code.

7. Wire the display: SPI1 (PA5=SCK, PA7=MOSI), PA4=CS, PA3=DC, PA2=RST.
8. Configure SPI1: CPOL=0 CPHA=0, 16-bit data frame, baud = APB2/2 = 42 MHz.
9. Write ILI9341 init sequence (or ST7789 depending on your module): send the ~15 config commands via blocking SPI, then set MADCTL for landscape if preferred.
10. Implement tft_set_window(x0,y0,x1,y1) → sends CASET + RASET + RAMWR commands.
11. Configure DMA2 Stream3 Channel3 (SPI1_TX): memory → peripheral, 16-bit, no-increment on peripheral, increment on memory, transfer complete + half-complete IRQs enabled.
12. Implement tft_push_strip_dma(uint16_t *buf, uint32_t len): call tft_set_window for the strip Y range, then start DMA.
13. In DMA half-complete IRQ: signal render task to begin filling the second half of the buffer (or the idle strip buffer in double-buffer mode).
14. Benchmark: push a solid-colour 240×160 strip and verify transfer time ≈ 7 ms at 42 MHz.

Step 3 — Offline Asset Conversion Pipeline (T04, T05)

Do this on your PC before porting any game code. Baked C arrays are the only practical asset storage.

15. Obtain Wolf3D shareware data (VSWAP.WL1, GAMEMAPS.WL1, MAPHEAD.WL1) — legally available.
16. Write `extract_vswap.py`: parse VSWAP header (`num_chunks`, `offsets`), extract 64×64 wall chunks 0–48, apply the game's 256-entry EGA palette, convert each pixel to RGB565 (`R>>3 | G>>2<<5 | B>>3<<11`), output as a C `uint16_t` array in `textures.c`.
17. Write `extract_map.py`: read MAPHEAD + GAMEMAPS, select level 1 plane 0, RLEW-decompress then Carmack-decompress, output as a 64×64 `uint16_t` array in `map_e1m1.c`.
18. Constrain texture array: keep only wall IDs actually used in your chosen level.
19. Add `__attribute__((section(".textures"), used))` to all texture arrays — place them in Flash.
20. Total Flash used by textures + map: target < 180 KB. Run `arm-none-eabi-size` on your `.elf` to verify.

Step 4 — Port the Raycaster (T06, T07, T08)

This is the largest and most critical engineering task. Work in a desktop simulator first (SDL2 on Linux/Windows), then cross-compile.

21. Create a minimal platform abstraction header: `plat.h` defines `plot_pixel(x, y, color)`, `get_ms()`, and the map accessor `get_tile(x, y)`.
22. Copy `WL_DRAW.C` and `WL_SCALE.C` into your project. Comment out all DOS/VGA-specific code blocks.
23. Replace the outer rendering loop to iterate 0..239 (screen columns) instead of 0..319.
24. Scale all fixed-point angles appropriately — Wolf3D uses a 3600-unit circle, this maps cleanly.
25. Rewrite `DrawWallColumn()`: remove the dynamic-code-gen call, replace with the integer C loop shown in Section 4.1.3. Use the Flash texture pointer directly.
26. Implement floor/ceiling: fill top and bottom bands with two solid RGB565 colours (grey ceiling, dark floor) — no texture lookup needed.
27. Build palette: convert the 256-entry Wolf3D EGA palette to a `uint16_t[256]` RGB565 lookup table, stored in Flash.
28. Test on desktop SDL2 simulator first — render to an SDL surface. Fix all issues before targeting hardware.
29. Cross-compile for STM32, write output to the strip buffer, push via DMA. Verify a static scene renders correctly.

Step 5 — Input System: Analogue Joystick + BlackPill KEY (T09, T10)

Your full input hardware is: analogue joystick (X/Y axes + click switch) and the BlackPill on-board KEY button (PA0). This maps cleanly to all essential Wolf3D actions.

5.1 Pin Assignment & Action Mapping

Input	GPIO / Peripheral	Wolf3D Action	Notes
Joystick Y-axis	PA1 / ADC1_IN1	Move forward / back	Analogue — proportional speed scaling

Input	GPIO / Peripheral	Wolf3D Action	Notes
Joystick X-axis	PA2 / ADC1_IN2	Turn left / right	Analogue — proportional turn rate
Joystick click	PB0 / GPIO EXTI	Fire / shoot	Digital, active LOW, pull-up, debounce 10 ms
BlackPill KEY	PA0 / GPIO EXTI	Open door / interact	Digital, active LOW, built-in, debounce 10 ms

5.2 ADC Configuration for Joystick Axes

Use ADC1 in scan mode with DMA to read both axes in one burst — never poll each channel separately.

30. In CubeMX: enable ADC1, add IN1 (PA1) and IN2 (PA2) to the scan sequence, set resolution to 12-bit, enable continuous conversion + DMA request.
31. Configure DMA2 Stream0 Channel0 (ADC1): peripheral → memory, 16-bit, circular mode, length 2. Target: volatile uint16_t adc_buf[2].
32. Trigger the ADC from TIM3 TRGO at 1 kHz — this gives clean, timer-synchronized samples without busy-waiting.
33. Read adc_buf[0] for X-axis, adc_buf[1] for Y-axis. Values are 0–4095; centre is nominally 2048.

5.3 Dead Zone & Proportional Speed

Raw ADC values at centre are noisy. Apply a dead zone, then scale to player speed linearly:

```
#define JOY_CENTRE    2048
#define DEAD_ZONE     200    // ±200 counts (~10% of range)
#define MAX_SPEED    0x6000  // fixed-point units/tick (tune to feel)
#define MAX_TURN     0x400   // fixed-point angle units/tick

static int16_t apply_deadzone(uint16_t raw) {
    int16_t v = (int16_t)raw - JOY_CENTRE;
    if (v > -DEAD_ZONE && v < DEAD_ZONE) return 0;
    return v; // signed: negative = back/left, positive = fwd/right
}

// In game tick handler:
int16_t joy_y = apply_deadzone(adc_buf[1]);
int16_t joy_x = apply_deadzone(adc_buf[0]);
player.speed = (joy_y * MAX_SPEED) / 2048; // proportional fwd/back
player.turn  = (joy_x * MAX_TURN) / 2048;  // proportional turn
```

5.4 Digital Inputs — Debounce

Both the joystick click and PA0 KEY need software debounce. A simple shift-register pattern is reliable and cheap:

```
// Call input_tick() every 10 ms from TIM2 IRQ
static uint8_t fire_hist = 0, door_hist = 0;

void input_tick(void) {
    // Shift history, sample current pin (active LOW → invert)
```

```

    fire_hist = (fire_hist << 1) | (!HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_0));
    door_hist = (door_hist << 1) | (!HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0));
}

// Stable press = last 4 samples all 1
#define PRESSED(hist)  (((hist) & 0x0F) == 0x0F)
// Rising edge = just became pressed
#define JUST_PRESSED(hist)  (((hist) & 0x1F) == 0x0F)

```

5.5 Input Abstraction Layer

Wrap all hardware reads behind a single input.h interface so game logic never touches registers directly. This also makes desktop simulation trivial (swap in SDL2 keyboard):

```

// input.h
typedef struct {
    int32_t move;    // fixed-point: positive=forward, negative=back
    int32_t turn;    // fixed-point: positive=right, negative=left
    uint8_t fire;    // 1 = held, use JUST_PRESSED for single shot
    uint8_t door;    // 1 on rising edge only (open door once per press)
} InputState;

void input_init(void); // configure ADC, DMA, GPIOs
void input_tick(void); // call every 10 ms — updates debounce history
InputState input_read(void); // call once per game frame — returns snapshot

```

5.6 PA0 Conflict Warning

PA0 is also ADC1_IN0. Critical: do NOT include channel 0 in your ADC scan sequence. Configure PA0 purely as a GPIO_Input with pull-up in CubeMX — if you accidentally leave it as Analog mode the KEY button will float and the ADC scan may produce garbage on nearby channels.

5.7 Port WL_PLAY.C Movement

34. Replace all keyboard boolean checks (Keyboard[sc_UpArrow] etc.) with InputState fields from input_read().
35. Forward/back: player.x += FixedMul(player.speed, costable[player.angle]); player.y += FixedMul(player.speed, sintable[player.angle]);
36. Turn: player.angle = (player.angle + player.turn) & (ANGLES - 1);
37. Collision: test 4 corners of player bounding box — if get_tile(px, py) >= WALL_TILE, undo the move.
38. Door open: on door_hist rising edge, check tile directly in front of player; if it is a door tile, call OpenDoor(tile_index).
39. Test: walk the full map perimeter, verify no wall clipping; test door open/close cycle; verify fire triggers the shoot animation frame.

Step 6 — Doors, Sprites, Enemy AI (T11, T12, T13)

40. Port door state machine from WL_ACT1.C: doors are special tiles (0x5A–0x65 range) with an open/close/wait timer. Each door has a byte state and a position (0=closed, 64=open).
41. In the raycaster, check if a ray hits a door tile — if so, offset the ray half a tile and look up the door texture; door texture X is modulated by door position to reveal the opening.

42. Sprite rendering: after wall pass, sort visible sprites by distance (insertion sort — max 32 sprites is fast enough). For each sprite column that is closer than the wall, write its pixel to the strip buffer.
43. Enemy AI: port one guard type from WL_ACT2.C. The FSM has 8 states (stand, path, chase, shoot, pain, die, dead). Each state has an animation frame list and a transition function. Timer-driven at game tick rate.

Step 7 — Profiling & Optimisation Loop (T15, T16)

Only optimise what profiling proves is slow. Do not guess.

44. Enable DWT cycle counter: `CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; DWT->CYCCNT = 0; DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;`
45. Wrap raycaster with cycle counter: `uint32_t t0 = DWT->CYCCNT; render(); uint32_t cycles = DWT->CYCCNT - t0;`
46. At 100 MHz, a 14 ms budget = 1,400,000 cycles. Log how many the raycaster uses.
47. Common hot spots and fixes:
 - Texture fetch from Flash (cache miss) → pre-copy 8–16 hot textures to SRAM texture cache at level load
 - Division in column height calc → replace with reciprocal lookup table: `wallHeight = (1 << 16) / distance`
 - sin/cos table miss → ensure `sin_table[]` is 4-byte aligned and fits within 1 KB for I-cache locality
 - Strip buffer write → use `uint32_t` writes (two pixels at once) to halve memory transactions
48. Compile raycaster with `-O2 -ffast-math -funroll-loops`. Verify no floating-point operations sneak into the inner loop (check disassembly).
49. If still over budget: reduce horizontal resolution to 120 px wide (every other column duplicated) — halves raycaster work.

Step 8 — HUD and Final Integration (T14)

50. Reserve the bottom 160 rows of the display (240×160) for the status bar.
51. Store a small 240×160 status bar bitmap in Flash (pre-rendered offline), blit it once per frame update.
52. Draw health and ammo as digit sprites from a small 8×8 font stored in Flash.
53. Final integration checklist before v1.0:
 - Raycaster + DMA pipeline: no tearing, no SRAM overrun (verify with `__stack_chk_guard` or MPU)
 - Enemy AI and player collision: no undefined behaviour (run with `-fsanitize=undefined` on desktop first)
 - Flash usage: `arm-none-eabi-size` shows < 480 KB (leave 32 KB headroom)
 - SRAM usage: `.bss + .data + stack watermark < 120 KB`
 - Frame rate: TIM2 tick shows ≥ 20 fps on target hardware with enemies present

7. Recommended Compiler & Linker Flags

```
# Compiler (arm-none-eabi-gcc)
```

```

-mcpu=cortex-m4 -mthumb -mfpv4-sp-d16 -mfloat-abi=hard
-O2 -ffast-math -ffunction-sections -fdata-sections
-Wall -Wextra -Wno-unused-parameter

# Inner raycaster loop only (separate TU):
-O3 -funroll-loops -fomit-frame-pointer

# Linker
--gc-sections -lm
-Wl,--print-memory-usage          # shows Flash + RAM used

```

8. Key Risks & Mitigations

Risk	Impact	Mitigation
SRAM exhausted by framebuffer + game state	HIGH	Use strip rendering; profile early; reduce actor limit to 16
Flash full before all assets fit	HIGH	4-bit palette textures (halve size); store only used walls per level
Raycaster too slow (< 15 fps)	MED	Reduce H-res to 120; use reciprocal LUT; enable SRAM texture cache
SPI bus too slow for smooth animation	MED	Push only changed strips; use DMA + double buffer; cap fps at 30
Fixed-point overflow on large maps	LOW	All coordinates use 22.10 fixed, max map 64×64 — safe with int32
Tile/sprite data licence compliance	MED	Use only shareware episode 1 data (freely distributable)

Wolf3D source: github.com/id-Software/wolf3d · Target: STM32F411CEU6 @ 100 MHz · Display: 2.4" SPI TFT 240×320
 Assets: use shareware Episode 1 data only (freely distributable). Non-shareware data requires a purchased licence.